

Evanesca Technical Report v0.1

EVANESCA PROJECT

Scope. Evanesca is a research prototype and open-source framework for reconstructing decentralized-finance (DeFi) transactions as Semantic Financial Graphs (SFGs) and auditing those graphs with transparent financial constraints. This technical report reorganizes the paper-era Evanesca work as a public whitepaper-style artifact. It explains the measurement problem, threat model, semantic recovery pipeline, graph abstraction, constraint methodology, evaluation snapshot, public examples, and release limitations.

Position. The framing is deliberately narrower than a production exploit detector. Evanesca surfaces constraint-backed candidate events and replayable evidence traces for analyst review; it does not infer malicious intent from traces alone. Using the IMC-version measurement snapshot, Evanesca analyzes 14,187,803 Ethereum transactions from 2020–2024, reconstructs 40 confirmed incidents across five chains, and flags 1,418 candidate instances, including 956 routine arbitrage baselines and 462 non-baseline instances. Source-code investigation of the non-baseline set identifies 219 instances in two under-studied defect families: reward-distribution manipulation and derivative-pricing/yield-token deviation.

1 Introduction

DeFi incidents are usually explained as isolated exploits: one protocol, one transaction, one postmortem. Yet many incidents share a common structure at the level of financial actions. A bZx-style price manipulation, a Harvest-style exchange-rate manipulation, a reward-distribution abuse, and a derivative-token pricing error may involve different contracts and event schemas, but each can be reconstructed as an ordered sequence of swaps, deposits, withdrawals, borrows, repayments, and profit-taking steps. The repeated structure suggests that value extraction can be measured systematically rather than investigated only case by case.

This observation creates a measurement challenge. Existing systems address important parts of the DeFi security problem: DeFiRanger and BlockEye detect known attack patterns or price-manipulation behavior [9, 7]; DeFort and DeFiScope focus on price-manipulation analysis with specialized logic [10, 11]; and operational systems such as Forta and OpenZeppelin Defender provide deployment-facing monitoring and alerting [3, 5]. These systems are useful, but they often bind analysis to a deployment-specific monitor, predefined attack template, or specific attack family. They also tend to emit alerts rather than the per-step financial computation needed for independent replay and triage.

Evanesca takes a different position. It is a measurement and forensic reconstruction framework. It recovers protocol-specific traces into uniform financial actions, constructs a Semantic Financial Graph, evaluates service-class financial constraints, and exports a replayable evidence trace. Each match is a *candidate event*, not a verdict that the transaction was malicious or harmful. This distinction matters because DeFi value extraction does not divide neatly into attacks and normal behavior. MEV and routine arbitrage can move value without a software defect [2, 8, 1], while flash-loan and oracle incidents can reuse similar swap and borrow primitives to exploit a protocol-local economic assumption [6, 13].

Report scope. This report is not a resubmission of the IMC short paper and is not meant to replace a future conference manuscript. It is a public technical report for the open-source release. The empirical numbers in the evaluation section are the IMC-version measurement snapshot unless explicitly labeled as validation or release evidence. Figures are reused from IMC, DeFi 26, and arXiv-era Evanesca drafts when they explain the system or a case study; non-IMC scale-out claims from older drafts are intentionally excluded.

Contributions. The public release centers on five concrete contributions.

- **A protocol-independent forensic abstraction.** Evanesca maps heterogeneous DeFi traces into typed financial

actions and then into SFGs. The novelty claim is not that graphs are new; it is that action recovery, graph construction, constraint evaluation, and evidence export are tied together as a reproducible measurement workflow.

- **Transparent constraint-based measurement.** Evanesca uses nine constraints in 107 lines of DSL to encode service-class financial assumptions rather than incident-specific signatures.
- **A scoped empirical snapshot.** The IMC-version evaluation analyzes 14,187,803 Ethereum transactions and separates routine arbitrage from non-baseline candidate families.
- **Replayable evidence artifacts.** The public release exports JSON evidence traces for bZx and Harvest case studies and includes manifests for larger public hash-set artifacts.
- **An open-source release path.** The release includes source code, documentation, public artifact validation, example datasets, and explicit limitations so that future work can extend the measurement pipeline without overstating the current prototype.

2 Definitions and Threat Model

2.1 Terminology

Evanesca separates measurement signals from final security judgments.

Candidate event.

A transaction or trace for which at least one financial constraint fires. A candidate requires review; it is not automatically an exploit.

Benign arbitrage baseline.

Value extraction that resolves price discrepancies across markets without evidence of violating a protocol-local accounting, oracle, collateral, reward, bridge, or derivative-pricing rule.

Harmful extraction.

Abnormal value transfer caused by exploiting or violating a protocol-local economic rule, such as a collateral relation, exchange-rate calculation, reward-allocation rule, bridge backing invariant, or oracle assumption.

Confirmed incident.

A publicly documented event with replayable on-chain transaction evidence and an external narrative. Confirmed incidents are used as validation anchors, not as a complete population sample.

2.2 Adversary Model

The actor may use one or more externally owned accounts or contracts, compose multiple DeFi protocols within one transaction, borrow temporary capital via a flash loan, manipulate an AMM or oracle-dependent state, trigger reward logic, and unwind positions before transaction completion. The actor may be a routine arbitrageur, MEV searcher, liquidator, or attacker. Evanesca does not require private mempool access, block-builder control, or sandwich capability as a core assumption; those behaviors can be represented when visible in traces, but the primary reconstruction unit is the transaction-local financial action sequence.

2.3 Loss Subjects

The loss subject depends on the service class. Evanesca distinguishes protocol reserves, liquidity providers, lending depositors, reward-pool participants, derivative-token holders, bridge backing reserves, and traders receiving worse execution. Some profitable transactions have no direct harmed protocol and are best described as market-efficiency arbitrage. This is why routine arbitrage is kept as an explicit baseline rather than collapsed into false positives.

Design position. Evanesca measures replayable financial-action patterns. It does not infer real-world identity, intent, or legal culpability. The output is evidence for analysts, not an automated accusation.

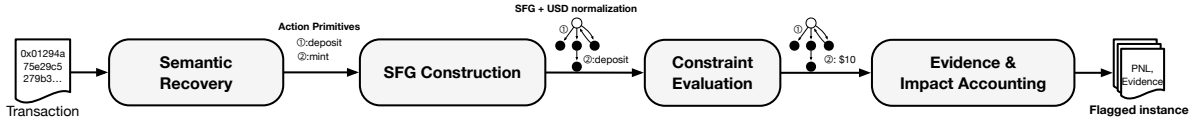


Figure 1: Evanesca pipeline: semantic recovery, SFG construction, constraint evaluation, and evidence export. The figure is reused from paper-era Evanesca drafts as system documentation.

3 Measurement Questions

The paper-era evaluation was organized around three research questions that remain useful for the public report.

- **RQ1: Incident reconstruction.** Can general-purpose constraints reconstruct known DeFi incidents without per-incident logic?
- **RQ2: Pattern landscape.** What candidate value-extraction patterns emerge at scale once benign arbitrage is separated from non-baseline behavior?
- **RQ3: Defect validation.** Do non-baseline candidates correspond to concrete implementation defects under source-code and event-log review?

The public release adds a fourth practical question: can the system expose enough code, data, and documentation for another researcher to inspect how a candidate was reconstructed? This report is structured around that reproducibility requirement.

4 System Overview

Figure 1 shows the pipeline. Each stage has a defined input and output, so failures can be localized: semantic recovery may miss an unsupported protocol, SFG construction may lack token metadata, constraint evaluation may fire too broadly, and evidence export may expose an incomplete diagnostic.

Semantic recovery. Protocol-specific adapters decode traces and logs into uniform financial actions. The public implementation covers common DEX, lending, flash-loan, and bridge families, but adapter coverage remains incomplete. The adapter boundary is deliberate: protocol-specific decoding belongs in semantic recovery, while constraints should operate over service-class financial actions.

SFG construction. Recovered actions are assembled into a directed graph $G = (V, E)$. Vertices represent accounts, pools, markets, protocol contracts, and token entities. Edges represent typed financial actions and carry token amounts, token addresses/symbols when known, USD-normalized values, protocol context, ordering, and raw evidence pointers.

Constraint evaluation. Constraints encode financial assumptions common to a service class, such as value preservation across swaps, collateral sufficiency, exchange-rate consistency, oracle sanity, reward-allocation stability, and bridge backing. Constraints are measurement hypotheses. A match is a candidate event.

Evidence export. The public CLI exports JSON containing transaction identity, a summary, graph nodes and edges, fired constraints, evidence steps, and diagnostics. The schema is documented in `docs/public-api/analysis-result.schema.md`; examples live in `docs/example-datasets/`.

5 Semantic Financial Graph

The SFG abstraction is designed to bridge the semantic gap between raw token transfers and financial interpretation. Transfer-only representations are often insufficient for DeFi incidents because profit can depend on collateral, share-price accounting, reward entitlement, virtual reserves, oracle reads, or derivative exchange rates. SFG edges therefore represent typed financial actions rather than only ERC-20 transfer events.

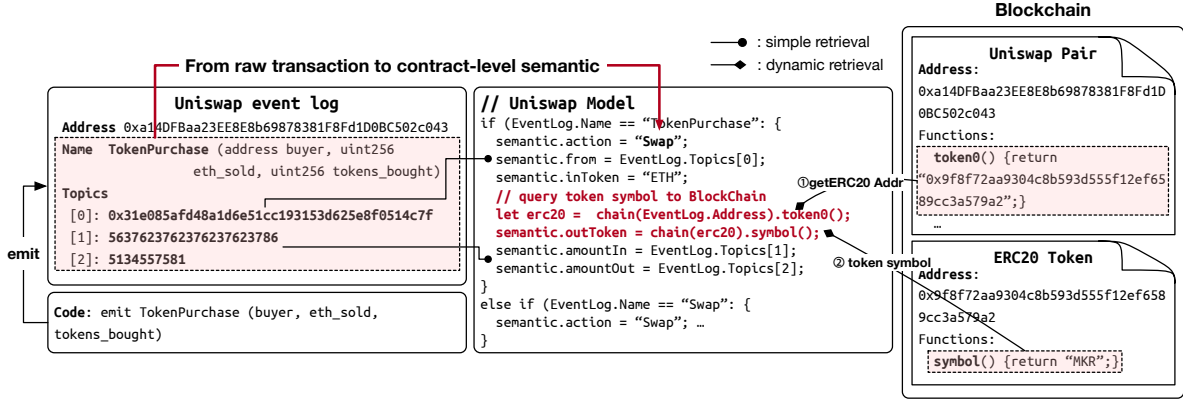


Figure 2: Semantic action model. Protocol-specific adapters map heterogeneous event schemas into uniform financial actions used by SFG construction.

Table 1: SFG elements exposed in the public evidence format.

Element	Meaning
Node	Account, protocol contract, pool, market, service, or token entity.
Edge	Typed financial action such as swap, borrow, repay, deposit, withdraw, bridge, or reward.
Value fields	Token amount, token symbol/address when known, and USD-normalized value.
Ordering	Transaction-local sequence used for deterministic reconstruction.
Diagnostics	Missing metadata, unsupported action classes, and recovery caveats.

Difference from cash-flow trees. DeFiRanger constructs cash-flow trees from raw transfer and balance-change patterns [9]. That is a strong fit for a subset of price manipulation attacks, but transfer-level templates struggle when the important state is a share price, collateral relation, reward trigger, or bridge backing rule. Evanesca does not claim that SFGs are categorically better for every task. Rather, SFGs are the representation Evanesca needs to make cross-service constraints and per-step PnL attribution replayable.

6 Constraint Methodology

For an edge e with nonzero USD input value, Evanesca commonly computes:

$$\rho(e) = \frac{\text{valueUSD}(\text{out}(e))}{\text{valueUSD}(\text{in}(e))}.$$

The paper-era implementation used $\alpha = 0.05$ for DEX swap edges and $\beta = 0.02$ for lending or oracle-mediated stablecoin-derivative edges. The DEX threshold is above reported routine unexpected slippage on Uniswap [12]; the stablecoin-derivative threshold is tighter because oracle studies report smaller normal movements for major stable assets [4]. These thresholds are measurement parameters, not universal constants. A stronger future paper should include sensitivity analysis over α , β , market regime, and protocol class.

Table 2: Current constraint families and service-class signals.

Constraint family	Service-class signal
PRICE_MANIPULATION	Abnormal swap value ratio or price deviation.
DEX_K_INVARIANT	AMM reserve-invariant deviation or significant swap imbalance.
LENDING_COLLATERALIZATION	Borrow, withdraw, or accounting behavior inconsistent with collateral assumptions.

Constraint family	Service-class signal
EXCHANGE_RATE_MANIPULATION	Vault, receipt-token, or derivative exchange-rate anomaly.
ORACLE_MANIPULATION	Oracle-mediated price/accounting inconsistency.
BRIDGE_INTEGRITY_VIOLATION	Bridge backing, mint, or redeem inconsistency.
FLASH_LOAN_ATTACK	Atomic temporary-capital pattern coupled with profit or manipulated state.
CONCENTRATED_LIQUIDITY_ATTACK	Tick, liquidity, or price-impact anomaly in concentrated-liquidity markets.
EMPTY_MARKET_ATTACK	Low-liquidity lending-market share-price anomaly.

DSL shape. Each constraint declares when it applies, which intermediate variables it derives, and which violation predicate fires. A simplified example is:

```
constraint PRICE_MANIPULATION {
  when: edge.Type == "DEX" && edge.Action == "Swap"
  condition: {
    price_ratio: edge.totalOutUSD / edge.totalInUSD
  }
  violation:
    edge.totalInUSD > 0 && price_ratio > 1.05
}
```

Production constraints add accounting guards and service-specific fields when available, but the key property is auditable replay: the predicate, derived values, and evidence edge are exported together.

7 Datasets and Scope

7.1 Scale-Out Dataset

The IMC-version scale-out evaluation covers 14,187,803 Ethereum transactions from January 2020 through December 2024 that match Evanescence’s supported adapter set. The remaining chains in the paper-era evaluation (BSC, Arbitrum, Avalanche, and Optimism) contribute confirmed-incident validation examples, not RQ2/RQ3 population prevalence estimates. Counts in this report should therefore be read as Ethereum scale-out results unless explicitly stated otherwise.

7.2 Confirmed-Incident Validation Set

The 40 confirmed incidents are publicly documented events with available transaction traces across the supported chains. A future polished paper should publish a full inclusion/exclusion protocol. The intended inclusion criteria are:

- public postmortem, incident report, or external narrative exists;
- exploit or representative transaction hash is available;
- on-chain trace/log data is accessible;
- economic loss or value transfer is attributable to on-chain behavior;
- the target chain and protocol family are supported by the collector.

Excluded categories include off-chain compromises without meaningful DeFi interaction, rug pulls without reconstructable financial actions, unsupported non-EVM chains, bridge-validator compromises without transaction-local financial reconstruction, and unavailable traces.

Table 3: Validation evidence retained for the public report.

Evaluation item	Result	Scope qualifier
Confirmed-incident reconstruction	40/40	Curated public-incident set; validates reconstruction criteria.
DeFiRanger within-scope comparison	12/17 vs. 17/17	Scope-aware PMA comparison from paper-era analysis.
Baseline arbitrage audit	956 reviewed	IMC baseline separation; not a population false-positive-rate claim.

Table 4: IMC-version flagged-instance breakdown.

Category	Count	Interpretation
Routine DEX arbitrage baseline	956	Profitable price-discrepancy resolution without harmful-evidence criteria.
Reward-distribution manipulation	207	Momentary in/out behavior around reward logic.
Price manipulation	166	Flash-loan or state-coupled abnormal price movement.
Yield-bearing token deviation	12	Cross-protocol derivative or receipt-token exchange-rate gap.
Other non-baseline candidates	77	Long tail requiring analyst review.
Total	1,418	Constraint-backed candidate instances.

7.3 Public Example Datasets

The public release includes a synthetic schema example and two replayed transaction examples:

- `docs/example-datasets/bzx-analysis-result.json`;
- `docs/example-datasets/harvest-analysis-result.json`.

These files are evidence-format examples, not a complete benchmark. The bZx example is byte-stable in the current public fallback environment. The Harvest example is schema-stable but provider-sensitive: public fallback RPC can miss Curve token metadata that configured archive/API providers recover, changing profit-attribution diagnostics while preserving the graph/evidence shape.

8 Evaluation Snapshot

8.1 RQ1: Confirmed-Incident Reconstruction

Evanesca reconstructs all 40 confirmed incidents under the paper-era success criteria: the evidence trace identifies the manipulation or value-extracting step, traces a profit path to the beneficiary, and fires at least one general-purpose constraint. This is validation evidence for reconstruction, not a claim that the incident set is an unbiased sample of all DeFi attacks.

8.2 RQ2: Pattern Landscape at Scale

The IMC-version scale-out evaluation flags 1,418 instances. The key measurement choice is to separate 956 routine arbitrage instances from 462 non-baseline instances. Treating the baseline as false positives would misstate the measurement task; treating all candidates as harmful would also be wrong.

Finding (RQ2). At 14.2M-transaction scale, the same nine constraints produce a structured landscape rather than a single attack template: 956 routine arbitrage and 462 non-baseline candidates spanning reward, price, yield-token, and long-tail families.

8.3 RQ3: Implementation-Defect Validation

The paper-era source-code and event-log review identifies 219 instances in two defect families. The 12 yield-bearing token deviations map to derivative-pricing or exchange-rate errors. The 207 reward-distribution instances map to reward-allocation bias where payout logic uses momentary balances rather than sustained exposure.

Derivative pricing and yield-token deviation. These cases involve cross-protocol deposit/withdraw or redemption paths where a receipt token, vault share, or derivative asset is priced inconsistently across services. The extraction is not visible as a single suspicious transfer; it appears when the SFG links acquisition, conversion, redemption, and unwind steps.

Reward-distribution bias. These cases use temporary liquidity or momentary balances to capture rewards without sustained holding. The public report keeps the numeric finding from the IMC snapshot but avoids claiming current exploitability or active disclosure status beyond what the public artifacts can substantiate.

Finding (RQ3). Of the 462 non-baseline instances, 219 correspond to concrete implementation-defect families under source-code/event-log review. The remaining non-baseline candidates include price manipulation and unresolved long-tail cases, so the public release should not describe every candidate as a vulnerability.

9 Case Studies

9.1 bZx Replay

The public bZx artifact is generated with:

```
npm run analyze -- --tx \  
0xb5c8bd9430b6cc87a0e2fe110ece6bf527fa4f170a4bc8cd032f768fc5219838 \  
--chain ethereum --out docs/example-datasets/bzx-analysis-result.json --pretty
```

In the checked-in public export, the transaction produces 10 graph nodes, 8 graph edges, 4 constraint records, and 8 evidence steps. The summary status is `flagged`; the category remains `unattributed`. This is appropriate for a public evidence-format example because the JSON demonstrates reconstruction and constraint evidence without overstating automatic classification.

9.2 Harvest Replay

The public Harvest artifact is generated with:

```
npm run analyze -- --tx \  
0x35f8d2f572fcea9288e5d462117850ef2694786992a8c3f6d02612277b0877 \  
--chain ethereum --out docs/example-datasets/harvest-analysis-result.json --pretty
```

The checked-in export produces 5 graph nodes, 18 graph edges, 2 constraint records, and 18 evidence steps. The summary reports exchange-rate manipulation signals and a flash-loan-profit diagnostic, while the category remains `unattributed`. The artifact is useful as an exchange-rate walkthrough and frontend evidence-trace input; it should not be cited as a final automatic incident label.

10 Comparison With Prior Work

The main difference is practical rather than rhetorical. Evanesca should be judged by whether it exports enough evidence for replay and triage: typed actions, graph edges, constraint predicates, derived values, and diagnostics. Prior systems remain important baselines for future conference work. A future submission should include a case-by-case comparison against DeFiRanger, DeFiScope, DeFort, Forta-style rules, and an ensemble baseline where feasible.

11 Public Release and Reproducibility

The open-source release is organized around artifacts rather than paper-only claims.

Table 5: Scope comparison with related DeFi security systems.

System	Primary input	Main abstraction	Relevance to Evanesca
DeFiRanger [9]	Traces/events	Cash-flow tree	Strong baseline for price-manipulation scope.
BlockEye [7]	On-chain events	Attack hunting	Operational attack hunting and monitoring.
DeFort [10]	Transaction/state data	PMA analysis	Specialized automatic price-manipulation detection.
DeFiScope [11]	DeFi operations	PMA reasoning workflow	Broad price-manipulation reasoning.
Forta/OZ [3, 5]	Live deployment events	Rule alerts	Operational monitoring and alert delivery.
Evanesca	Traces and semantic actions	SFG plus constraints	Replayable candidate reconstruction across service classes.

Table 6: Public release artifact status.

Artifact	Status
Technical report	This LaTeX/PDF report and the companion Markdown planning note.
Source code	Public repository with CLI smoke path; repository-wide TypeScript build cleanup remains open.
Documentation	README, source-code guide, public claims policy, release checklist, and reproducibility guide.
Reproducible artifacts	Public artifact manifests and validator; population reproduction still requires external RPC/API services.
Example datasets	Synthetic schema sample plus replayed bZx and Harvest JSON exports.

Minimum public commands.

```

npm install
npm run analyze -- --help
npm run analyze -- --tx <transaction-hash> --chain ethereum \
  --out out/analysis.json --pretty
npm run validate:public-artifacts

```

Claim policy. Every public empirical claim should identify the code revision, input hashes or dataset manifest, command line, configuration, external API/RPC requirements, and expected output. Claims that require private credentials or unpublished datasets must be marked as unreproduced in the public release.

12 Limitations

Evanesca remains a research prototype. The main limitations are:

- **Adapter coverage.** Unsupported protocols can produce false negatives or incomplete evidence traces.
- **Trace and RPC dependence.** Public fallback RPC may omit token metadata or historical state needed for richer diagnostics.
- **USD normalization sensitivity.** Historical price sources can affect profit diagnostics, especially for thinly traded or derivative tokens.
- **Threshold sensitivity.** α and β are defended measurement parameters, not universal constants.

- **Manual labeling.** The public release does not yet provide a complete precision/recall evaluation over the 1,418 candidate set.
- **Build status.** The source tree has a public CLI smoke path and artifact validation, but repository-wide TypeScript build cleanup remains a release gate.

13 Ethics and Disclosure

The release should avoid publishing exploit recipes beyond already public transaction traces and high-level financial-action explanations. The public examples use historical transactions and focus on reconstruction, not live attack enablement. Candidate labels must be phrased conservatively: flagged transactions are evidence for review, not accusations. Any future disclosure of newly active defects should follow coordinated disclosure norms before public release.

14 Future Work

A future conference submission remains optional. If pursued, it should not reuse this technical report as-is; it should add the missing evaluation rigor:

- explicit inclusion/exclusion criteria for the 40 incident set;
- semantic-recovery correctness evaluation with sampled manual labels;
- candidate classification protocol with multiple annotators;
- precision/recall or top- k triage evaluation;
- threshold sensitivity analysis for α and β ;
- case-by-case comparison against prior systems and ensembles;
- held-out protocol evaluation to quantify adapter generality.

15 Conclusion

Evanescence exposes DeFi transactions as replayable financial-action evidence rather than opaque alerts. This report reframes the paper-era prototype as a public open-source technical artifact: a source tree, SFG/evidence schema, constraint methodology, IMC-scoped measurement snapshot, replayed examples, and reproducibility path. The right success criterion for the release is not whether every candidate is automatically labeled, but whether another researcher can inspect how a candidate was reconstructed and decide what claim is justified.

References

- [1] Crypto Briefing. What is MEV? ethereum’s invisible tax explained. <https://cryptobriefing.com/what-is-mev-ethereums-invisible-tax-explained/>, 2021.
- [2] Flashbots. Flashbots: Illuminating the dark forest. <https://www.flashbots.net/>, 2024.
- [3] Forta Foundation. Forta: A decentralized runtime security solution for automated threat detection and prevention. <https://docs.forta.network/>, 2022.
- [4] Bowen Liu, Pawel Szalachowski, and Jianying Zhou. A first look into DeFi oracles. In *IEEE International Conference on Decentralized Applications and Infrastructures*, pages 39–48. IEEE, 2021.
- [5] OpenZeppelin. OpenZeppelin defender: Smart contract monitoring with rule-based sentinels. <https://docs.openzeppelin.com/defender/module/monitor>, 2024.
- [6] Kaihua Qin, Liyi Zhou, Benjamin Livshits, and Arthur Gervais. Attacking the DeFi ecosystem with flash loans for fun and profit. In *International Conference on Financial Cryptography and Data Security*, pages 3–32. Springer, 2021.

- [7] Bin Wang, Han Liu, Chao Liu, Zhiqiang Yang, Qian Ren, Huixuan Zheng, and Hong Lei. BLOCKEYE: Hunting for DeFi attacks on blockchain. In *IEEE/ACM International Conference on Software Engineering: Companion Proceedings*, pages 17–20. IEEE, 2021.
- [8] Ben Weintraub, Christof Ferreira Torres, Cristina Nita-Rotaru, and Radu State. A flash(bot) in the pan: Measuring maximal extractable value in private pools. In *Proceedings of the ACM Internet Measurement Conference*, pages 458–471. ACM, 2022.
- [9] Siwei Wu, Zhou Yu, Dabao Wang, Yajin Zhou, Lei Wu, Haoyu Wang, and Xingliang Yuan. DeFiRanger: Detecting DeFi price manipulation attacks. *IEEE Transactions on Dependable and Secure Computing*, 21(4):4147–4161, 2024.
- [10] Maoyi Xie, Ming Hu, Ziqiao Kong, Cen Zhang, Yebo Feng, Haijun Wang, Yue Xue, Hao Zhang, Ye Liu, and Yang Liu. DeFort: Automatic detection and analysis of price manipulation attacks in DeFi applications. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1517–1529. ACM, 2024.
- [11] Juantao Zhong, Daoyuan Wu, Ye Liu, Maoyi Xie, Yang Liu, Yi Li, and Ning Liu. DeFiScope: Detecting various DeFi price manipulations with LLM reasoning. *IEEE Transactions on Dependable and Secure Computing*, 2025.
- [12] Liyi Zhou, Kaihua Qin, Christof Ferreira Torres, Duc Viet Le, and Arthur Gervais. High-frequency trading on decentralized on-chain exchanges. In *IEEE Symposium on Security and Privacy*, pages 428–445. IEEE, 2021.
- [13] Liyi Zhou, Xihan Xiong, Jens Ernstberger, Stefanos Chaliasos, Zhipeng Wang, Ye Wang, Kaihua Qin, Roger Wattenhofer, Dawn Song, and Arthur Gervais. Sok: Decentralized finance (DeFi) attacks. In *IEEE Symposium on Security and Privacy*, pages 2444–2461. IEEE, 2023.

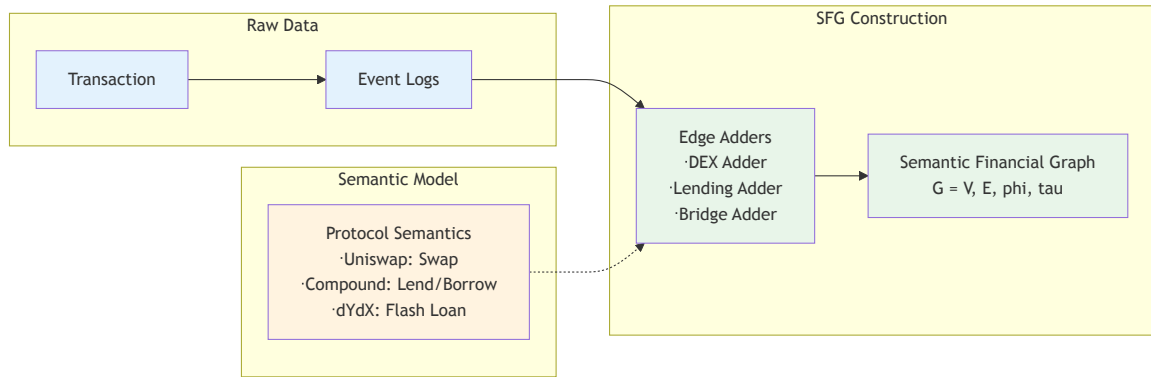


Figure 3: SFG construction. Edges preserve typed actions, value fields, and ordering information needed for replayable evidence.

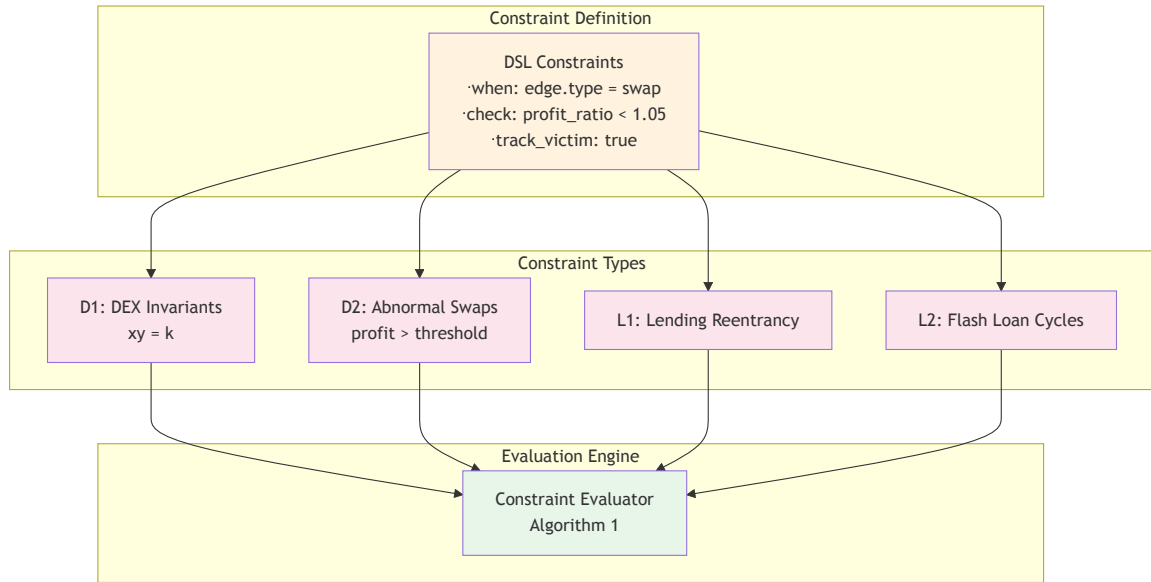


Figure 4: DSL constraint overview. Constraints encode service-class financial assumptions over SFG edges and produce replayable evidence records.

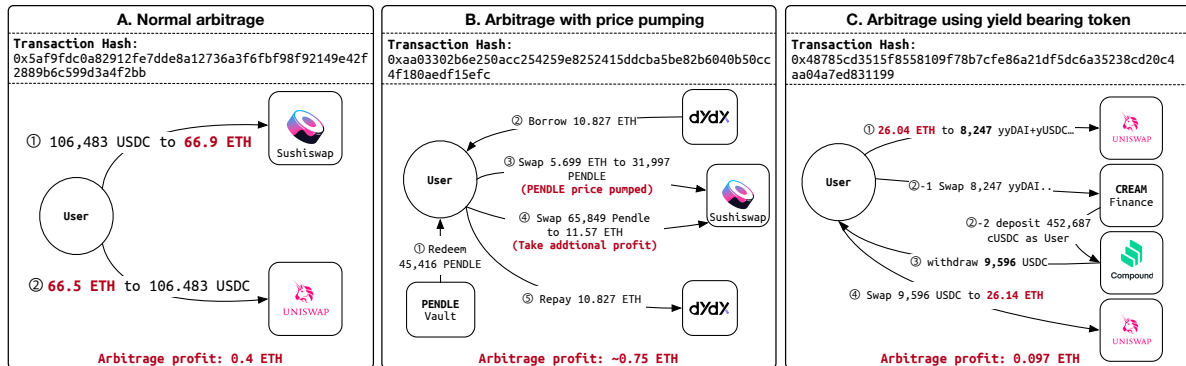


Figure 5: Arbitrage and non-baseline patterns. Panel A is retained as a benign baseline; Panels B and C show manipulation and yield-token paths that require additional review.

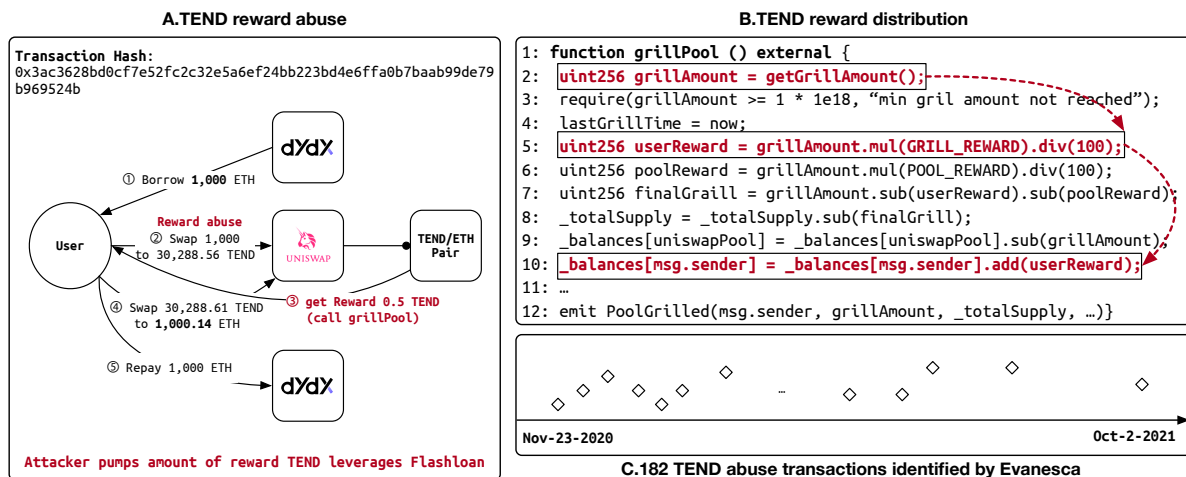


Figure 6: Representative reward-distribution manipulation pattern. A flash-loan-funded path triggers reward payout and immediately unwinds.

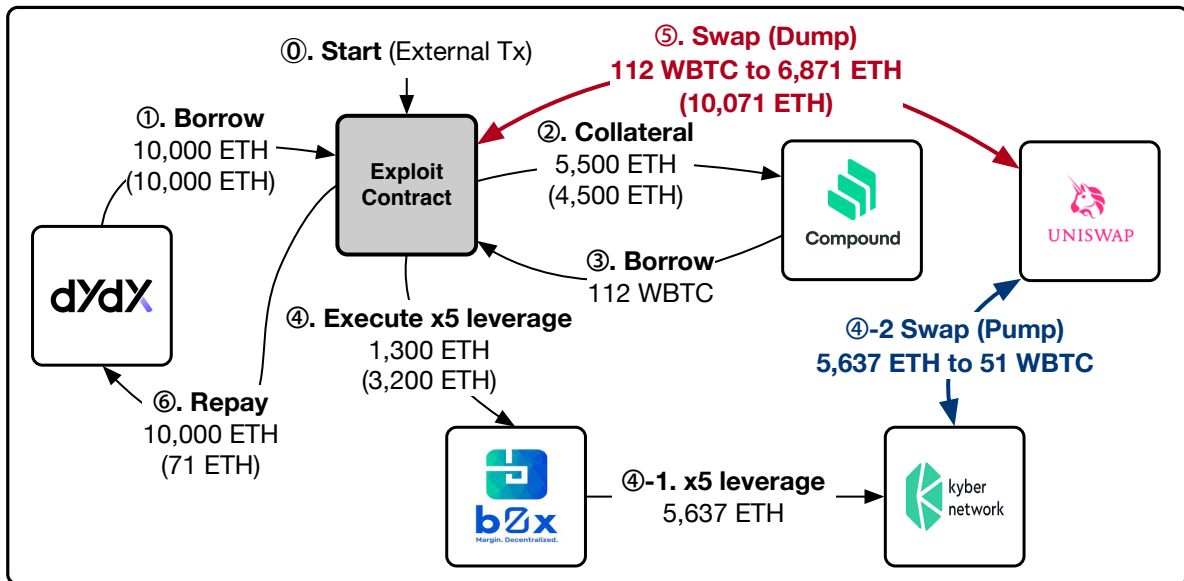


Figure 7: Representative bZx-style price-manipulation path used as a walkthrough for semantic recovery, SFG construction, and evidence extraction.